ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

AI23 (23TNT1), FIT@HCMUS, VNUHCM

Đồ án 3 - Stochastic Methods

Đề tài: Thuật toán tối ưu hóa - Phương pháp ngẫu nhiên

Môn học: Cơ sở trí tuệ nhân tạo - TTNT2023

Sinh viên thực hiện:
Đinh Đức Anh Khoa (23122001)
Nguyễn Lê Hoàng Trung (23122004)
Đinh Đức Tài (23122013)

Giáo viên hướng dẫn:
GS.TS Lê Hoài Bắc
ThS. Lê Nhựt Nam

Ngày 30 tháng 12 năm 2025



Mục lục

1 Giới thiệu **3**

2 Tối ưu hóa bằng phương pháp ngẫu nhiên **4**
2.1 Noisy Descent . . . 4
2.1.1 Kỹ thuật Noisy Descent . . . 4
2.1.2 Stochastic gradient descent . . . 4
2.2 Mesh Adaptive Direct Search . . . 5
2.2.1 Tổng quan về Mesh Adaptive Direct Search . . . 5
2.2.2 Kỹ thuật lấy mẫu theo hướng . . . 5
2.2.3 Cập nhật bước nhảy . . . 5
2.2.4 Thuật toán MADS . . . 6
2.3 Memory-Efficient Zeroth-Order Optimization . . . 6
2.4 Simulated Annealing . . . 7
2.5 Cross-Entropy Method . . . 7
2.5.1 Các thành phần cơ bản . . . 8
2.5.2 Quy trình thực hiện . . . 8
2.6 Natural Evolution Strategies . . . 9
2.6.1 Ước lượng Gradient . . . 10
2.6.2 Các thành phần cơ bản . . . 10
2.6.3 Quy trình thực hiện . . . 11
2.7 Covariance Matrix Adaptation . . . 12
2.7.1 Các thành phần cơ bản . . . 12
2.7.2 Quy trình thực hiện . . . 12
2.7.3 Covariance Matrix Adaptation . . . 13
2.7.4 Hằng số khuyên dùng . . . 15

3 Thực nghiệm **15**
3.1 Noisy Descent . . . 15
3.1.1 Bài toán Rastrigin . . . 15
3.1.2 Phân tích kết quả thực nghiệm . . . 16
3.1.3 Kết luận . . . 16
3.2 Mesh Adaptive Direct Search . . . 17
3.3 MeZO . . . 18
3.3.1 Bài toán Rosenbrock . . . 18
3.3.2 Phân tích kết quả thực nghiệm . . . 18
3.3.3 Kết luận . . . 19
3.4 Simulated Annealing . . . 20
3.4.1 Bài toán Rastrigin . . . 20
3.4.2 Phân tích kết quả thực nghiệm . . . 20
3.4.3 Kết luận . . . 21
3.5 Cross-Entropy Method . . . 22
3.6 Natural Evolution Strategies . . . 24
3.7 Covariance Matrix Adaptation . . . 26

4 Các nghiên cứu hiện đại liên quan đến phương pháp tối ưu hóa ngẫu nhiên **27**
4.1 Particle Swarm Optimization 27
4.2 GaLore: Gradient Low-Rank Projection 28
4.3 SOAP: Improving and Stabilizing Shampoo using Adam 28

5 Kết luận, đánh giá **29**
5.1 Kết luận 29
5.2 Đánh giá 29

1 Giới thiệu

Đây là tài liệu báo cáo cho **Đồ án 3 - Stochastic Methods**, môn **Cơ sở trí tuệ nhân tạo - TTNT2023**, Khoa Công nghệ thông tin, Trường Đại học Khoa học tự nhiên - Đại học Quốc gia TP.HCM. Chủ đề của đồ án là trình bày các thuật toán tối ưu hóa theo phương pháp ngẫu nhiên.

Báo cáo được thực hiện bởi nhóm các thành viên:

- Đinh Đức Anh Khoa (23122001)
- Nguyễn Lê Hoàng Trung (23122004)
- Đinh Đức Tài (23122013)

Đường dẫn repository Github: https://github.com/anhKhoa585/AI-Fundamentals-Proj3 [1]

Bảng phân công nhiệm vụ cho từng thành viên:

Họ và tên	MSSV	Nhiệm vụ
Đinh Đức Anh Khoa	23122001	- Trình bày phần MeshAdaptive Direct Search - Trình bày phần Covariance Matrix Adaptation. - Viết code thực nghiệm cho các phần tương ứng.
Nguyễn Lê Hoàng Trung	23122004	- Trình bày Cross-Entropy Method - Trình bày Natural Evolution Strategies - Viết code thực nghiệm cho các phần tương ứng.
Đinh Đức Tài	23122013	- Trình bày Noisy Descent, MeZO, Simulated Annealing - Viết code minh họa cho các thuật toán trên - Báo cáo các nghiên cứu hiện đại liên quan đến phương pháp tối ưu hóa ngẫu nhiên

Các công nghệ sử dụng:

- **Python**: Python là một ngôn ngữ lập trình bậc cao, đa năng. Python được sử dụng rộng rãi trong các lĩnh vực như phát triển web, khoa học dữ liệu, trí tuệ nhân tạo, và tự động hóa.
- **NumPy**: NumPy là thư viện mã nguồn mở hỗ trợ tính toán số học hiệu năng cao, tập trung vào đối tượng mảng n-chiều (ndarray) cùng các thuật toán đại số tuyến tính và biến đổi tích phân. NumPy tối ưu hóa việc xử lý dữ liệu thông qua cơ chế vector hóa, giúp giảm thiểu độ trễ tính toán so với các vòng lặp truyền thống.
- **Matplotlib**: Matplotlib là một thư viện lập trình mã nguồn mở phổ biến trong ngôn ngữ Python, được dùng để tạo ra các loại biểu đồ và hình ảnh chất lượng cao từ dữ liệu.

2 Tối ưu hóa bằng phương pháp ngẫu nhiên

Stochastic Methods - các phương pháp ngẫu nhiên sử dụng phép ngẫu nhiên một cách có chiến lược để giúp khám phá không gian nhằm tìm ra điểm tối ưu. Sự ngẫu nhiên này có thể giúp ta thoát khỏi các điểm tối ưu cục bộ và tăng cơ hội tìm thấy điểm tối ưu toàn cục. Tính ngẫu nhiên quá lớn thường không hiệu quả vì nó ngăn cản chúng ta sử dụng tốt các điểm được đánh giá trước đó để hướng dẫn quá trình tìm kiếm. Vì vậy, cần phải có nhiều cách để kiểm soát mức độ ngẫu nhiên trong quá trình tìm kiếm này.

Các kiến thức trong tài liệu này được trình bày lại từ sách *Algorithms for optimization* [2] - MIT.

2.1 Noisy Descent

2.1.1 Kỹ thuật Noisy Descent

Noisy Descent là kỹ thuật cải tiến phương pháp *Gradient Descent* bằng cách thêm tính ngẫu nhiên vào quá trình cập nhật tham số. Việc thêm tính ngẫu nhiên vào phương pháp *Gradient Descent* có thể mang lại lợi ích trong các bài toán tối ưu hóa phi tuyến tính quy mô lớn. Các điểm yên ngựa, nơi mà gradient rất gần bằng không, có thể khiến các phương pháp hạ gradient chọn kích thước bước quá nhỏ để có thể hiệu quả. Một phương pháp để giải quyết vấn đề này là thêm nhiễu Gaussian vào mỗi bước hạ $\mathbf{d}^{(k)}$:

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \mathbf{d}^{(k)} + \epsilon^{(k)} \tag{1}$$

trong đó $\epsilon^{(k)}$ là nhiễu Gaussian có kỳ vọng bằng không với độ lệch chuẩn σ. Lượng nhiễu thường được giảm dần theo thời gian. Độ lệch chuẩn của nhiễu thường là một dãy giảm $\sigma^{(k)}$, ví dụ như $1/k$.

2.1.2 Stochastic gradient descent

Một phương pháp phổ biến để huấn luyện các mạng neuron là *stochastic gradient descent* (hạ gradient ngẫu nhiên). Phương pháp này sử dụng một xấp xỉ gradient có nhiễu. Ngoài việc giúp vượt qua các điểm yên ngựa, việc đánh giá các gradient có nhiễu bằng cách sử dụng các tập con được chọn ngẫu nhiên từ dữ liệu huấn luyện (batch) ít tốn kém về mặt tính toán hơn đáng kể so với việc tính toán gradient thực ở mỗi vòng lặp.

Để đảm bảo hội tụ, *stochastic gradient descent* yêu cầu các kích thước bước phải được chọn sao cho:

$$\sum_{k=1}^{\infty} \alpha^{(k)} = \infty, \quad \sum_{k=1}^{\infty} \left(\alpha^{(k)}\right)^2 < \infty \tag{2}$$

Các điều kiện này đảm bảo rằng kích thước bước giảm dần và cho phép phương pháp hội tụ, nhưng không giảm quá nhanh đến mức bị kẹt ở xa một điểm cực tiểu cục bộ.

2.2 Mesh Adaptive Direct Search

2.2.1 Tổng quan về Mesh Adaptive Direct Search

Mesh adaptive direct search (MADS) là một phương pháp tối ưu hóa ngẫu nhiên (stochastic method), được phát triển từ các phương pháp tìm kiếm mẫu tổng quát (generalized pattern search). Các phương pháp tìm kiếm mẫu truyền thống thường chỉ thăm dò cục bộ trong một tập hợp các hướng cố định (ví dụ như chỉ đi dọc theo các trục tọa độ). Điều này có thể khiến thuật toán bị kẹt hoặc di chuyển rất chậm nếu hướng tối ưu không nằm trong tập hướng cố định đó. Thay vì dùng hướng cố định, MADS sử dụng *random positive spanning directions* (tạm dịch là các hướng bao trùm tích cực ngẫu nhiên). Điều này cho phép thuật toán tiếp cận bất kỳ hướng nào trong không gian thiết kế khi số lần lặp đủ lớn.

2.2.2 Kỹ thuật lấy mẫu theo hướng

Để đảm bảo khả năng tìm kiếm bao quát không gian $\mathbb{R}^n$, MADS sinh ra một tập hợp các hướng gọi là *Positive Spanning Set*. Quy trình sinh hướng dựa trên ma trận tam giác dưới ngẫu nhiên được thực hiện như sau:

Bước 1: Tại bước lặp k với tham số bước lưới $\alpha^{(k)}$, ta xây dựng ma trận L. Các phần tử đường chéo được chọn ngẫu nhiên từ tập:

$$L_{ii} \in \left\{ \pm \frac{1}{\sqrt{\alpha^{(k)}}} \right\} \tag{3}$$

Các phần tử nằm dưới đường chéo được lấy mẫu ngẫu nhiên trong khoảng số nguyên:

$$L_{ij} \in \left\{ -\frac{1}{\sqrt{\alpha^{(k)}}} + 1, \ldots, \frac{1}{\sqrt{\alpha^{(k)}}} - 1 \right\} \quad \text{với } j < i \tag{4}$$

Việc chuẩn hóa bằng $\sqrt{\alpha^{(k)}}$ giúp điều chỉnh độ lớn và sự đa dạng của các vector khi bước lưới thay đổi.

Bước 2: Để tránh bias thứ tự biến, các hàng và cột của L được hoán vị ngẫu nhiên để tạo thành ma trận D chứa n hướng độc lập tuyến tính. Để có thể tạo thành một tập bao trùm dương (positive spanning set), ta thêm hướng thứ $n+1$ là đối của tổng các hướng đã có:

$$d^{(n+1)} = -\sum_{i=1}^{n} d^{(i)} \tag{5}$$

Mỗi hướng này nằm trong một siêu lập phương có cạnh dài $2/\sqrt{\alpha^{(k)}}$.

2.2.3 Cập nhật bước nhảy

Tham số kích thước bước α kiểm soát độ mịn của lưới tìm kiếm. α luôn là một lũy thừa của 4 và không vượt quá 1. Quy tắc cập nhật được thực hiện như sau:

$$\alpha^{(k+1)} \leftarrow \begin{cases} \min(1, 4\alpha^{(k)}) & \text{nếu tìm thấy điểm tốt hơn} \\ \alpha^{(k)}/4 & \text{nếu không tìm thấy} \end{cases} \tag{6}$$

Khi thành công, bước nhảy tăng lên để mở rộng vùng tìm kiếm. Khi thất bại, bước nhảy giảm đi để tìm kiếm chi tiết hơn.

2.2.4 Thuật toán MADS

Thuật toán hoạt động theo cơ chế cơ hội (opportunistic), nghĩa là ngay khi tìm thấy một hướng giúp cải thiện hàm mục tiêu, thuật toán sẽ chấp nhận bước đi đó ngay lập tức mà không cần đợi kiểm tra hết tất cả các hướng còn lại. Quy trình thực hiện của thuật toán:

1. **Khởi tạo:** Điểm x, bước nhảy $\alpha = 1$, dung sai ϵ.
2. **Vòng lặp chính (khi $\alpha > \epsilon$):**
   - Sinh tập hướng ngẫu nhiên D (gồm $n + 1$ hướng).
   - Duyệt qua từng hướng d trong tập D:
     - Tạo điểm thử nghiệm: $x' = x + \alpha d$.
     - Kiểm tra cải thiện: Nếu $f(x') < f(x)$:
       * Cập nhật điểm hiện tại: $x \leftarrow x'$.
       * **Mở rộng:** Thử điểm $x'' = x + 3\alpha d$. Nếu x'' tốt hơn x', cập nhật $x \leftarrow x''$.
       * Đánh dấu là đã cải thiện và thoát vòng lặp hướng để sang bước cập nhật α.
     - Cập nhật α:
       * Nếu có cải thiện: $\alpha \leftarrow \min(1, 4\alpha)$
       * Nếu không: $\alpha \leftarrow \alpha/4$.
3. **Kết thúc:** Trả về điểm x tốt nhất tìm được.

Vì các hướng được sinh ngẫu nhiên và thay đổi liên tục qua các vòng lặp, MADS có xác suất cao sẽ thăm dò được hướng dẫn đến cực trị toàn cục nếu chạy đủ lâu.

2.3 Memory-Efficient Zeroth-Order Optimization

Hiệu quả bộ nhớ trở thành một mối quan tâm lớn khi tối ưu hóa các thiết kế rất lớn, chẳng hạn như các mô hình học sâu với hàng tỷ tham số (gần tới giới hạn của khả năng lưu trữ bộ nhớ). Các phương pháp ngẫu nhiên không chỉ thực tiễn mà còn có thể hữu ích vì việc tính toán chính xác một gradient có thể cực kỳ tốn kém hoặc không khả thi. *Memory-efficient Zeroth-order optimizer - MeZO* (bộ tối ưu hóa bậc không tiết kiệm bộ nhớ) tính toán một bước gradient ngẫu nhiên có thể được ước lượng và áp dụng tại chỗ, sử dụng cùng một dung lượng bộ nhớ dùng để lưu trữ vectơ thiết kế. Điều này cho phép tối ưu hóa các bài toán lớn hơn nhiều.

Cách tiếp cận này sử dụng phương pháp xấp xỉ gradient ngẫu nhiên nhiễu loạn đồng thời[1] (SPSA). SPSA xấp xỉ gradient bằng cách sử dụng các đạo hàm theo hướng dọc theo các hướng được chọn ngẫu nhiên. Trong *MeZO*, gradient được ước lượng và một bước cập nhật được áp dụng tại chỗ sao cho không cần cấp phát một vectơ gradient riêng biệt, nhờ đó giảm thiểu dấu chân bộ nhớ. Cập nhật được thực hiện như sau:

$$\mathbf{x}' = \mathbf{x} - \alpha \frac{f(\mathbf{x} + \delta\mathbf{z}) - f(\mathbf{x} - \delta\mathbf{z})}{2\delta}\mathbf{z} \tag{7}$$

[1] Simultaneous perturbation stochastic approximation

với hệ số bước α, đại lượng vô hướng sai phân hữu hạn δ, và sự nhiễu loạn $\mathbf{z}$ được lấy từ phân phối chuẩn tắc Gaussian. Một lần cập nhật như vậy được gọi là một *zero-order stochastic step* (bước ngẫu nhiên bậc không).

2.4 Simulated Annealing

Simulated Annealing (giả lập luyện kim) mượn cảm hứng từ lĩnh vực luyện kim. Trong luyện kim, *nhiệt độ* được sử dụng để kiểm soát mức độ biến đổi ngẫu nhiên trong quá trình tìm kiếm. *Nhiệt độ* bắt đầu ở mức cao, cho phép quá trình tự do di chuyển khắp không gian tìm kiếm, với hy vọng rằng trong giai đoạn này, quá trình sẽ tìm thấy một vùng tốt chứa điểm cực tiểu toàn cục. Sau đó, *nhiệt độ* được hạ xuống từ từ, giảm bớt tính ngẫu nhiên và buộc quá trình tìm kiếm phải hội tụ về một điểm cực tiểu. *Simulated Annealing* thường được sử dụng cho các hàm có nhiều cực tiểu cục bộ do khả năng thoát khỏi các cực tiểu này của nó.

Tại mỗi vòng lặp, một *bước chuyển đề xuất* từ $\mathbf{x}$ sang $\mathbf{x}'$ được lấy mẫu từ một phân phối chuyển đổi T và được chấp nhận với xác suất:

$$P(\text{chấp nhận}) = \begin{cases} 1 & \text{nếu } \Delta y \leq 0 \\ e^{-\Delta y/t} & \text{nếu } \Delta y > 0 \end{cases} \tag{8}$$

trong đó $\Delta y = f(\mathbf{x}') - f(\mathbf{x})$ là sự chênh lệch về giá trị mục tiêu và t là *nhiệt độ*. Xác suất chấp nhận này được gọi là *tiêu chuẩn Metropolis*, cho phép thuật toán thoát khỏi các cực tiểu cục bộ khi *nhiệt độ* còn cao.

Tham số nhiệt độ t kiểm soát xác suất chấp nhận. Một *annealing schedule* (lịch trình hạ nhiệt) được sử dụng để hạ dần *nhiệt độ* khi thuật toán tiến triển. *Nhiệt độ* phải được hạ xuống đủ chậm để đảm bảo hội tụ. Nếu hạ nhiệt quá nhanh, phương pháp tìm kiếm có thể không bao phủ được phần không gian chứa điểm cực tiểu toàn cục.

Có thể chứng minh rằng một *logarithmic annealing schedule* (lịch trình hạ nhiệt logarit):

$$t^{(k)} = t^{(1)} \frac{\ln(2)}{\ln(k+1)} \tag{9}$$

đảm bảo hội tụ tiệm cận đến điểm tối ưu toàn cục dưới một số điều kiện nhất định, nhưng nó rất chậm trong thực tế. *Exponential annealing schedule* (lịch trình hạ nhiệt hàm mũ) phổ biến hơn, sử dụng một hệ số suy giảm đơn giản $t^{(k+1)} = \gamma t^{(k)}$ với $\gamma \in (0, 1)$. Một lịch trình phổ biến khác là *fast annealing* (hạ nhiệt nhanh), sử dụng nhiệt độ $t^{(k)} = t^{(1)}/k$.

2.5 Cross-Entropy Method

Cross-Entropy Method (CEM) là một phương pháp tối ưu hóa ngẫu nhiên (stochastic method), được phát triển từ việc mô phỏng các sự kiện hiếm (rare-event simulation) và ước lượng mật độ xác suất trong mạng lưới stochastic phức tạp thông qua việc cực tiểu hóa phương sai. Thay vì duy trì một điểm giải pháp đơn lẻ, CEM duy trì một phân phối xác suất tường minh trên không gian tìm kiếm, được gọi là phân phối đề xuất (proposal distribution).

Phương pháp này hoạt động dựa trên nguyên lý cập nhật các tham số của phân phối đề xuất sao cho khoảng cách Kullback–Leibler (Cross-Entropy) giữa phân phối hiện tại và phân phối tối ưu lý thuyết là nhỏ nhất. CEM nổi bật nhờ sự đơn giản về mặt toán học và khả năng hội tụ toàn cục tốt mà không cần phải đạo hàm của hàm mục tiêu trong nhiều bài toán phức tạp.

2.5.1 Các thành phần cơ bản

Phương pháp CEM yêu cầu xác định một họ phân phối xác suất $p(\cdot;\theta)$ được tham số hóa bởi vector θ. Đối với các bài toán tối ưu hóa liên tục, lựa chọn phổ biến nhất là phân phối chuẩn Gauss đa biến, với các tham số chính:

- **Vector trung bình (μ):** Đại diện cho vị trí trung tâm của phân phối tìm kiếm, nơi mật độ xác suất tập trung cao nhất.
- **Ma trận hiệp phương sai (Σ):** Xác định hình dạng ellipsoid của phân phối (độ phân tán theo các hướng khác nhau).[2]
- **Hệ số làm mịn(α):** Làm mịn tham số để đảm bảo sự ổn định số học, việc sử dụng α giúp thuật toán giữ lại phần lớn thông tin từ quá khứ, tránh việc thay đổi phân phối quá đột ngột, từ đó duy trì khả năng tìm kiếm được cực trị toàn cục. [3]

Trong quá trình tối ưu, các tham số $\theta = (\mu, \Sigma)$ sẽ liên tục được cập nhật sao cho phân phối $p(\cdot;\theta)$ tập trung toàn bộ mật độ xác suất vào điểm cực trị toàn cục.

2.5.2 Quy trình thực hiện

Thuật toán hoạt động dựa trên cơ chế lấy mẫu và cập nhật lặp lại để hướng phân phối xác suất về phía cực trị toàn cục. Quy trình thực hiện như sau:

- **Lấy mẫu:** Tại mỗi bước lặp k, thuật toán sinh ra một quần thể gồm m mẫu thử nghiệm từ phân phối Gauss đa biến:
$$x \sim \mathcal{N}(\mu^{(k)}, \Sigma^{(k)}) \tag{10}$$
Kích thước mẫu m cần đủ lớn (thường lớn hơn số chiều bài toán) để đảm bảo tính đại diện cho không gian tìm kiếm hiện tại.
- **Chọn lọc mẫu tốt nhất:** Sau khi đánh giá giá trị hàm mục tiêu $f(x)$ cho tất cả các mẫu, các mẫu được sắp xếp theo thứ tự tốt nhất đến xấu nhất. Một tập hợp gồm m_{elite} mẫu tốt nhất sẽ được giữ lại để dùng cho việc cập nhật tham số. Tỷ lệ mẫu tốt nhất ($\rho = m_{elite}/m$) thường chọn trong khoảng $0.01 \leq \rho \leq 0.1$.
- **Cập nhật tham số:** Dựa trên tập mẫu tốt nhất, các tham số mới cho bước lặp tiếp theo được tính toán thông qua ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE):

[2]Khác với việc chỉ dùng một biến bước nhảy đơn lẻ, ma trận này cho phép thuật toán học được cấu trúc tương quan giữa các biến, giúp thích nghi với địa hình của hàm mục tiêu

[3]Trong thực tế, việc cập nhật trực tiếp các tham số μ và Σ hoàn toàn dựa trên các mẫu tốt nhất (MLE) thường khiến phân phối co lại quá nhanh, dẫn đến phương sai tiến về 0 trước khi tìm thấy cực trị toàn cục (hiện tượng hội tụ sớm).

– **Cập nhật vector trung bình μ:** Vector trung bình mới là trung bình cộng của các mẫu tốt nhất, giúp hướng phân phối về phía vùng có giá trị tốt hơn:

$$\mu^{(k+1)} = \frac{1}{m_{elite}} \sum_{i=1}^{m_{elite}} x^{(i)} \tag{11}$$

– **Cập nhật ma trận hiệp phương sai Σ:** Ma trận hiệp phương sai mới được tính dựa trên sai số giữa các mẫu tốt nhất và vector trung bình mới, giúp định hình lại phân phối để khớp với cấu trúc của vùng cực trị:

$$\Sigma^{(k+1)} = \frac{1}{m_{elite}} \sum_{i=1}^{m_{elite}} (x^{(i)} - \mu^{(k+1)})(x^{(i)} - \mu^{(k+1)})^\top \tag{12}$$

Trong đó:

* m_{elite}: Số lượng mẫu tốt nhất được chọn.
* $x^{(i)}$: Vectơ vị trí của mẫu tốt nhất thứ i trong không gian tìm kiếm.
* $\mu^{(k+1)}$: Vectơ trung bình mới, đại diện trung tâm của các mẫu tốt nhất.
* $(x^{(i)} - \mu^{(k+1)})(x^{(i)} - \mu^{(k+1)})^\top$: Tích ngoài của vector sai số, đóng vai trò là ma trận hiệp phương sai thành phần của mẫu thứ i.

– **Áp dụng kỹ thuật làm mịn**: Bằng cách kết hợp tham số của bước trước đó với tham số vừa ước lượng cùng với chỉ số làm mịn:

$$\mu^{(k+1)} = \alpha\mu^{(k)} + (1-\alpha)\mu^{k+1} \tag{13}$$
$$\Sigma^{(k+1)} = \alpha\Sigma^{(k)} + (1-\alpha)\Sigma^{k+1} \tag{14}$$

Trong đó: Hệ số làm mịn (α) thường chọn trong khoảng $0.7 \leq \alpha \leq 0.99$.

Quy trình này đảm bảo rằng ở mỗi lần lặp, phân phối đề xuất sẽ ngày càng tập trung hẹp hơn xung quanh các mẫu tốt nhất, dẫn đến sự hội tụ của thuật toán.

2.6 Natural Evolution Strategies

Giống như phương pháp CEM, Natural Evolution Strategies (NES) là một họ thuật toán tối ưu hóa ngẫu nhiên. Khác với phương pháp CEM, NES áp dụng phương pháp Gradient Descent (tạm dịch là suy giảm độ dốc) được ước lượng trực tiếp từ các mẫu để tối ưu hóa tham số $\boldsymbol{\theta}$ của phân phối xác suất $p(\cdot;\boldsymbol{\theta})$

Mục tiêu cốt lõi của NES là cực tiểu hóa giá trị kỳ vọng của hàm mục tiêu $f(x)$ dựa trên phân phối:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{x\sim p(\cdot|\theta)}[f(x)] \tag{15}$$

Bằng cách điều chỉnh tham số $\boldsymbol{\theta}$ để hướng phân phối về những vùng có giá trị hàm mục tiêu tốt hơn, thuật toán sẽ dần hội tụ về điểm cực trị toàn cục.

2.6.1 Ước lượng Gradient

Thủ thuật Log-Likelihood: Để tối ưu hóa $J(\theta)$ bằng phương pháp Gradient Descent, ta cần tính đạo hàm của hàm kỳ vọng mà không cần biết đạo hàm của chính hàm $f(x)$. Gradient của giá trị kỳ vọng đối với tham số θ được ước lượng như sau:

$$\begin{aligned}\nabla_\theta J(\theta) = \nabla_{\boldsymbol{\theta}} \mathbb{E}_{\mathbf{x}\sim p(\cdot|\boldsymbol{\theta})}[f(\mathbf{x})] &= \int \nabla_{\boldsymbol{\theta}} p(\mathbf{x}|\boldsymbol{\theta}) f(\mathbf{x}) d\mathbf{x} \\ &= \int \frac{p(\mathbf{x}|\boldsymbol{\theta})}{p(\mathbf{x}|\boldsymbol{\theta})} \nabla_{\boldsymbol{\theta}} p(\mathbf{x}|\boldsymbol{\theta}) f(\mathbf{x}) d\mathbf{x} \\ &= \int p(\mathbf{x}|\boldsymbol{\theta}) \nabla_{\boldsymbol{\theta}} \log p(\mathbf{x}|\boldsymbol{\theta}) f(\mathbf{x}) d\mathbf{x} \\ &= \mathbb{E}_{\mathbf{x}\sim p(\cdot|\boldsymbol{\theta})}[f(\mathbf{x}) \nabla_{\boldsymbol{\theta}} \log p(\mathbf{x}|\boldsymbol{\theta})]\end{aligned} \tag{16}$$

Trong thực tế, kỳ vọng này được xấp xỉ bằng phương pháp Monte Carlo, tính toán dựa trên trung bình cộng của m mẫu:

$$\nabla_\theta J(\theta) = \nabla_{\boldsymbol{\theta}} \mathbb{E}_{\mathbf{x}\sim p(\cdot|\boldsymbol{\theta})}[f(\mathbf{x})] \approx \frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \nabla_{\boldsymbol{\theta}} \log p(\mathbf{x}^{(i)}|\boldsymbol{\theta}) \tag{17}$$

Dù không cần gradient của hàm mục tiêu $f(x)$, phương pháp này yêu cầu phải tính được gradient của log-likelihood $\nabla_{\boldsymbol{\theta}} \log p(\mathbf{x}|\boldsymbol{\theta})$. Sau đó có thể sử dụng phương pháp Gradient Descent để cập nhật θ dựa trên gradient ước lượng.

2.6.2 Các thành phần cơ bản

Tương tự như phương pháp CEM, NES yêu cầu xác định một họ phân phối xác suất $p(\cdot; \boldsymbol{\theta})$ được tham số hóa bởi vector $\boldsymbol{\theta}$. Đối với các bài toán tối ưu hóa liên tục, lựa chọn phổ biến nhất là phân phối chuẩn Gauss đa biến $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, với các tham số chính:

- **Vectơ trung bình ($\boldsymbol{\mu}$):** Đại diện cho vị trí trung tâm của phân phối tìm kiếm, nơi mật độ xác suất tập trung cao nhất.
- **Ma trận hiệp phương sai ($\boldsymbol{\Sigma}$):** Xác định hình dạng ellipsoid của phân phối (độ phân tán và tương quan giữa các chiều).

Hàm mật độ xác suất (PDF) trong không gian n chiều được định nghĩa như sau:

$$p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \tag{18}$$

Log-likelihood của phân phối này là:

$$\log p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{n}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) \tag{19}$$

Để đảm bảo ma trận hiệp phương sai luôn đối xứng và xác định dương trong quá trình cập nhật, ta áp dụng kỹ thuật Cholesky Decomposition tham số hóa Σ thông qua một ma trận biến đổi $\mathbf{A}$

sao cho:

$$\boldsymbol{\Sigma} = \mathbf{A}^\top \mathbf{A} \tag{20}$$

Để áp dụng phương pháp NES, ta cần tính gradient của log-likelihood theo các tham số $\boldsymbol{\theta}$. Đối với phân phối Gauss, các thành phần gradient của log-likelihood được tính như sau:

- Gradient theo vectơ trung bình $\boldsymbol{\mu}$:

$$\nabla_{\boldsymbol{\mu}} \log p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \tag{21}$$

- Gradient theo ma trận hiệp phương sai $\boldsymbol{\Sigma}$:

$$\nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{2}\left(\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} - \boldsymbol{\Sigma}^{-1}\right) \tag{22}$$

- Gradient theo ma trận biến đổi $\mathbf{A}$: Sử dụng quy tắc chuỗi, gradient theo $\mathbf{A}$ được tính từ $\nabla_{\boldsymbol{\Sigma}}$

$$\nabla_{\mathbf{A}} \log p(\mathbf{x}) = \mathbf{A}\left(\nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x}) + \nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x})^\top\right) \tag{23}$$

Trong thực tế, ta xấp xỉ gradient của hàm mục tiêu $J(\boldsymbol{\theta})$ bằng trung bình mẫu trên m điểm dữ liệu được sinh ra từ phân phối hiện tại:

$$\begin{aligned}\nabla_{\boldsymbol{\mu}} J &\approx \frac{1}{m}\sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \nabla_{\boldsymbol{\mu}} \log p(\mathbf{x}^{(i)}) \\ &= \frac{1}{m}\sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \boldsymbol{\Sigma}^{-1}(\mathbf{x}^{(i)} - \boldsymbol{\mu})\end{aligned} \tag{24}$$

$$\begin{aligned}\nabla_{\boldsymbol{\Sigma}} J &\approx \frac{1}{m}\sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x}^{(i)}) \\ &= \frac{1}{2m}\sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \left(\boldsymbol{\Sigma}^{-1}(\mathbf{x}^{(i)} - \boldsymbol{\mu})(\mathbf{x}^{(i)} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} - \boldsymbol{\Sigma}^{-1}\right)\end{aligned} \tag{25}$$

Kỹ thuật ổn định (Damping): Để tránh việc ma trận hiệp phương sai bị suy biến (singular) khi phân phối co lại quá nhanh, ta sử dụng ma trận hiệp phương sai hiệu chỉnh khi tính nghịch đảo:

$$\hat{\boldsymbol{\Sigma}} = \boldsymbol{\Sigma} + \epsilon \mathbf{I} \tag{26}$$

Trong đó, $\mathbf{I}$ là ma trận đơn vị và hệ số điều chỉnh được chọn là $\epsilon = 0.2$. Ma trận $\hat{\boldsymbol{\Sigma}}$ này sau đó được sử dụng để tính toán các gradient $\nabla_{\boldsymbol{\mu}}$ và $\nabla_{\boldsymbol{\Sigma}}$.

2.6.3 Quy trình thực hiện

Lấy mẫu: Tại mỗi bước lặp k, thuật toán sinh ra một quần thể gồm m mẫu thử nghiệm từ phân phối hiện tại:

$$x \sim \mathcal{N}(\mu^{(k)}, \Sigma^{(k)}), \text{ với } \boldsymbol{\Sigma^{(k)}} = \mathbf{A^{(k)}}^\top \mathbf{A^{(k)}} \tag{27}$$

Ước lượng Gradient: Sau khi tính toán giá trị hàm mục tiêu $f(x)$ cho tất cả các mẫu, ước lượng giá trị gradient $\nabla_{\boldsymbol{\mu}} J, \nabla_{\mathbf{A}} J$

$$\nabla_{\boldsymbol{\mu}} J^{(k)} \approx \frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \boldsymbol{\Sigma}^{(k)^{-1}} (\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k)}) \tag{28}$$

$$\nabla_{\boldsymbol{\Sigma}} J^{(k)} \approx \frac{1}{2m} \sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \left(\boldsymbol{\Sigma}^{(k)^{-1}} (\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k)})^{\top} \boldsymbol{\Sigma}^{(k)^{-1}} - \boldsymbol{\Sigma}^{(k)^{-1}} \right) \tag{29}$$

$$\nabla_{\mathbf{A}} J^{(k)} = \mathbf{A} \left(\nabla_{\boldsymbol{\Sigma}} J^{(k)} + \nabla_{\boldsymbol{\Sigma}} J^{(k)^{\top}} \right) \tag{30}$$

Cập nhật tham số: Dựa trên gradient ước lượng, các tham số sẽ được cập nhật theo hướng ngược lại của gradient với tốc độ học $\boldsymbol{\alpha}$.

$$\boldsymbol{\mu}^{(k+1)} = \boldsymbol{\mu}^{(k)} - \alpha \nabla_{\boldsymbol{\mu}} J \tag{31}$$

$$\mathbf{A}^{(k+1)} = \mathbf{A}^{(k)} - \alpha \nabla_{\mathbf{A}} J \tag{32}$$

Quy trình này lặp lại cho đến tập trung toàn bộ mật độ xác suất vào điểm cực trị toàn cục, dẫn đến sự hội tụ của thuật toán.

2.7 Covariance Matrix Adaptation

Covariance Matrix Adaptation, thường được gọi là CMA-ES (Covariance Matrix Adaptation Evolution Strategy), là một phương pháp tối ưu hóa ngẫu nhiên hiệu quả và ổn định, thuộc nhóm *evolution strategies*. Phương pháp này duy trì và cập nhật một phân phối Gauss đa biến để sinh mẫu, từ đó dần dần thích nghi với hình dạng của hàm mục tiêu. So với phương pháp *cross-entropy* và *natural evolution strategies*, CMA-ES nổi bật nhờ khả năng tự động học cấu trúc hiệp phương sai đầy đủ, giúp khai thác các hướng có lợi trong không gian tìm kiếm.

2.7.1 Các thành phần cơ bản

Phương pháp CMA duy trì ba tham số chính để kiểm soát phân phối đề xuất:

1. **Vector trung bình (μ):** Đại diện cho vị trí trung tâm của phân phối tìm kiếm.
2. **Ma trận hiệp phương sai (Σ)**: Xác định hình dạng ellipsoid của phân phối.
3. **Hệ số kích thước bước nhảy (σ):** Kiểm soát độ rộng tổng thể của phân phối.

Trong mỗi bước lặp, CMA không chỉ cập nhật vị trí mà còn thay đổi hình dạng vùng tìm kiếm và quy mô tìm kiếm dựa trên thông tin thu được từ các mẫu.

2.7.2 Quy trình thực hiện

Lấy mẫu: Tại mỗi bước lặp, thuật toán tạo ra m mẫu thiết kế từ phân phối Gauss đa biến:

$$\mathbf{x} \sim \mathcal{N}(\mu, \sigma^2 \Sigma) \tag{33}$$

Số lượng mẫu thường được sử dụng là $4 + \lfloor 3 \ln n \rfloor$ với n là số chiều của bài toán.

Cập nhật vector trung bình: Sau khi đánh giá giá trị hàm mục tiêu $f(\mathbf{x})$ cho tất cả các mẫu, các mẫu được sắp xếp theo thứ tự tốt nhất đến xấu nhất, một vector trung bình mới $\mu^{(k+1)}$ được tạo thành bằng cách lấy trung bình có trọng số của m_{elite} mẫu tốt nhất (thường là một nửa số mẫu):

$$\mu^{(k+1)} \leftarrow \sum_{i=1}^{m_{elite}} w_i \mathbf{x}^{(i)} \tag{34}$$

Trong đó, các trọng số w_i dương, có tổng bằng 1 và giảm dần (theo quy luật logarit). Cách gán trọng số này làm cho CMA-ES cập nhật trung bình mạnh và định hướng hơn so với cross-entropy method.

Kiểm soát kích thước bước nhảy: CMA sử dụng một biến tích lũy $\mathbf{p}_\sigma$ gọi là *evolution path* (tạm dịch là đường tiến hóa) để theo dõi lịch sử các bước di chuyển nhằm điều chỉnh σ. Nguyên lý hoạt động như sau: Nếu các bước di chuyển liên tiếp cùng hướng, độ dài của đường dẫn $\mathbf{p}_\sigma$ sẽ dài ra và ta sẽ tăng kích thước bước σ để di chuyển nhanh hơn. Nếu các bước di chuyển triệt tiêu lẫn nhau, độ dài $\mathbf{p}_\sigma$ sẽ ngắn lại, ta sẽ giảm σ để hội tụ. Đường dẫn $\mathbf{p}_\sigma$ được cập nhật như sau:

$$\mathbf{p}_\sigma^{(k+1)} \leftarrow (1 - c_\sigma)\mathbf{p}_\sigma + \sqrt{c_\sigma(2 - c_\sigma)\mu_{eff}}(\mathbf{\Sigma}^{(k)})^{-1/2}\boldsymbol{\delta}_w \tag{35}$$

Trong đó:

- $c_\sigma < 1$ là tốc độ suy giảm (decay rate).
- μ_{eff} là Variance effective selection mass: $\mu_{eff} = \frac{1}{\sum_{i=1}^{m_{elite}} w_i^2}$
- $(\Sigma^{(k)})^{-1/2}$ là phép biến đổi để chuẩn hóa bước đi về hệ tọa độ đẳng hướng (isotropic).
- $\boldsymbol{\delta}_w$ được tính từ các độ lệch mẫu: $\delta_w = \sum_{i=1}^{m_{elite}} w_i \delta^{(i)}$ với $\delta^{(i)} = \frac{x^{(i)} - \mu^{(k)}}{\sigma^{(k)}}$

Kích thước bước σ sau đó được cập nhật bằng cách so sánh độ dài chuẩn (norm) của $\mathbf{p}_\sigma$ với kỳ vọng độ dài của một vector ngẫu nhiên chuẩn hóa $\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$:

$$\sigma^{(k+1)} \leftarrow \sigma^{(k)} \exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|} - 1\right)\right) \tag{36}$$

2.7.3 Covariance Matrix Adaptation

Việc cập nhật Σ dựa trên việc tối đa hóa khả năng (likelihood) của việc sinh ra các mẫu tốt. Gradient của log-likelihood đối với ma trận hiệp phương sai Σ được xác định là:

$$\nabla_{(\Sigma)} \log p(x|\mu, \Sigma) = \frac{1}{2}\Sigma^{-1}(x - \mu)(x - \mu)^\top \Sigma^{-1} - \frac{1}{2}\Sigma^{-1} \tag{37}$$

việc cập nhật trực tiếp theo gradient này không đảm bảo ma trận kết quả là xác định dương (yêu cầu bắt buộc của ma trận hiệp phương sai). Do đó, CMA-ES sử dụng phương pháp phân rã $\Sigma = A^\top A$ và cập nhật A để đảm bảo tính chất này.

Quá trình cập nhật Σ trong CMA-ES bao gồm hai cơ chế chính kết hợp với nhau: Cập nhật hạng một (Rank-one update) và Cập nhật hạng m_{elite} (Rank-m_{elite} update). Để thực hiện cập nhật hạng một, thuật toán duy trì một vector tích lũy p_Σ, gọi là đường dẫn tiến hóa. Vector này ghi nhớ lịch sử các bước di chuyển trung bình qua các thế hệ. Công thức cập nhật p_Σ:

$$p_\Sigma^{(k+1)} \leftarrow (1 - c_\Sigma)p_\Sigma^{(k)} + h_\sigma \sqrt{c_\Sigma(2 - c_\Sigma)\mu_{eff}}\delta_w \tag{38}$$

Trong đó:

- c_Σ: Hệ số suy giảm, kiểm soát lượng thông tin quá khứ được giữ lại.
- δ_w: Bước di chuyển trung bình có trọng số của quần thể hiện tại ($\sum w_i\delta^{(i)}$).
- h_σ: Hàm Heaviside (có giá trị 0 hoặc 1). Tác dụng của hàm này là ngắt việc cập nhật p_Σ nếu kích thước bước σ quá nhỏ nhưng p_Σ lại quá lớn, giúp ngăn chặn sự bùng nổ giá trị của ma trận Σ.

Đối với việc cập nhật ma trận hiệp phương sai, các trọng số cần được xử lý đặc biệt để đảm bảo tính ổn định, đặc biệt là với các trọng số âm:

$$w_i' = \begin{cases} w_i & \text{nếu } w_i \geq 0 \\ \frac{w_i}{||\Sigma^{-1/2}\delta^{(i)}||^2} & \text{trường hợp khác} \end{cases} \tag{39}$$

Kết hợp các thành phần trên, ta có công thức cập nhật tổng quát để tạo ra ma trận hiệp phương sai cho thế hệ tiếp theo:

$$\Sigma^{(k+1)} \leftarrow \underbrace{(1 - c_1 - c_\mu + \dots)\Sigma^{(k)}}_{\text{Phần cũ suy giảm}} + \underbrace{c_1 p_\Sigma p_\Sigma^\top}_{\text{Rank-one}} + \underbrace{c_\mu \sum_{i=1}^{m_{elite}} w_i'\delta^{(i)}(\delta^{(i)})^\top}_{\text{Rank-}m_{elite}} \tag{40}$$

Công thức chi tiết:

$$\Sigma^{(k+1)} \leftarrow (1 + c_1c_\Sigma(1 - h_\sigma)(2 - c_\Sigma) - c_1 - c_\mu)\Sigma^{(k)} + c_1 p_\Sigma p_\Sigma^\top + c_\mu \sum_{i=1}^{m_{elite}} w_i'\delta^{(i)}(\delta^{(i)})^\top \tag{41}$$

Giải thích thành phần:

1. **Thành phần suy giảm:** Hệ số nhân với $\Sigma^{(k)}$ đảm bảo rằng thông tin cũ sẽ giảm dần theo thời gian. Biểu thức $c_1c_\Sigma(1-h_\sigma)(2-c_\Sigma)$ là phần bù chính xác để xử lý trường hợp khi $h_\sigma = 0$ (khi việc cập nhật đường dẫn p_Σ bị ngắt).
2. **Cập nhật Hạng một (Rank-one update)**:
   - Sử dụng tích ngoài của vector đường dẫn p_Σ.
   - Ma trận $p_\Sigma p_\Sigma^\top$ có hạng bằng 1.
   - Nếu thuật toán di chuyển liên tục theo một hướng, thành phần này sẽ kéo dài ma trận hiệp phương sai theo hướng đó, giúp tăng tốc độ hội tụ.

3. **Cập nhật Hạng:**
    - Sử dụng tổng các tích ngoài của các vector bước nhảy $\delta^{(i)}$.
    - Ma trận này có hạng là $\min(m_{elite}, n)$.
    - Ước lượng phương sai dựa trên các bước nhảy thành công ngay trong thế hệ hiện tại.

2.7.4 Hằng số khuyên dùng

Công thức của CMA-ES có rất nhiều hằng số. Để tiết kiệm thời gian tinh chỉnh, các hằng số được khuyến nghị phụ thuộc vào số chiều không gian n và số mẫu hiệu quả $\mu_{eff} = 1/\sum w_i^2$ như sau:

1. **Hằng số cho kích thước bước σ:**
    - Tốc độ học: $c_\sigma = \frac{\mu_{eff}+2}{n+\mu_{eff}+5}$
    - Hệ số giảm chấn: $d_\sigma = 1 + 2\max\left(0, \sqrt{\frac{\mu_{eff}-1}{n+1}} - 1\right) + c_\sigma$
2. **Hằng số cho ma trận hiệp phương sai Σ:**
    - Tốc độ học đường dẫn: $c_\Sigma = \frac{4+\mu_{eff}/n}{n+4+2\mu_{eff}/n}$
    - Tốc độ học Rank-1: $c_1 = \frac{2}{(n+1.3)^2+\mu_{eff}}$
    - Tốc độ học Rank-μ: $c_\mu = \min\left(1 - c_1, 2\frac{\mu_{eff}-2+1/\mu_{eff}}{(n+2)^2+\mu_{eff}}\right)$
3. **Hằng số chuẩn hóa kỳ vọng:** $\mathbb{E}||\mathcal{N}(0, I)|| \approx \sqrt{n}\left(1 - \frac{1}{4n} + \frac{1}{21n^2}\right)$

3 Thực nghiệm

Trong phần này, nhóm thực hiện cài đặt và đánh giá các thuật toán tối ưu hóa ngẫu nhiên đã được trình bày ở phần lý thuyết. Các thuật toán được hiện thực hóa bằng ngôn ngữ Python, tuân thủ chặt chẽ theo mã giả và các nguyên lý được mô tả trong tài liệu.

3.1 Noisy Descent

3.1.1 Bài toán Rastrigin

Để minh họa khả năng vượt qua các điểm cực tiểu cục bộ của thuật toán *Noisy Descent* so với *Gradient Descent* truyền thống, ta thực hiện demo trên hàm *Rastrigin* 2 chiều. Đây là một hàm kiểm thử tối ưu hóa điển hình với đặc điểm có rất nhiều điểm cực tiểu cục bộ bao quanh điểm cực tiểu toàn cục tại $(0, 0)$. Các thông số bao gồm:

- Hàm mục tiêu: $f(x) = 10n + \sum_{i=1}^{n}[x_i^2 - 10\cos(2\pi x_i)]$ với $n = 2$.
- Điểm khởi đầu: $\mathbf{x}^{(0)} = (2.5, 2.5)$.
- Tốc độ học: $\alpha = 0.01$.

- Độ lệch chuẩn nhiễu ban đầu: $\sigma = 4.0$, áp dụng cơ chế giảm dần theo thời gian ($\sigma^{(k)} = \sigma/k$).
- Số vòng lặp tối đa: 300.

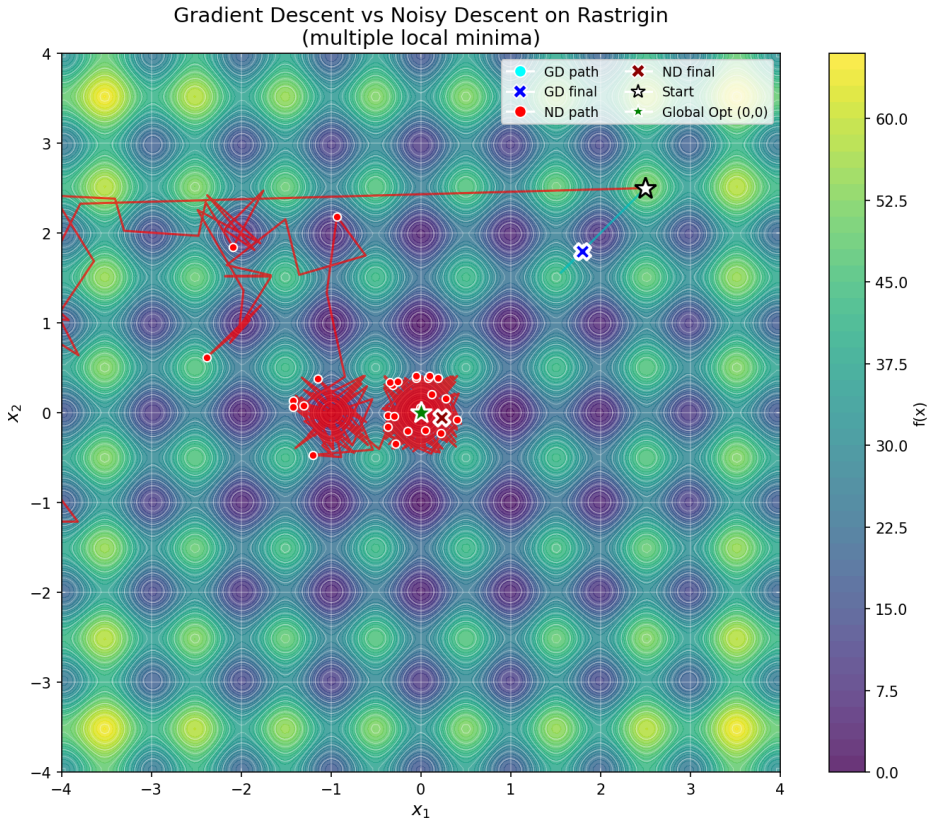
3.1.2 Phân tích kết quả thực nghiệm

Quỹ đạo di chuyển: Quan sát hình ảnh mô phỏng đường đi, phương pháp *Gradient Descent* (đường màu xanh cyan) nhanh chóng bị mắc kẹt tại điểm cực tiểu cục bộ gần điểm xuất phát nhất ($f(x) \approx 20.40$). Ngược lại, thuật toán *Noisy Descent* (đường màu đỏ) nhờ việc thêm nhiễu Gaussian đã thực hiện các bước nhảy ngẫu nhiên, giúp thực thể thoát khỏi các "hố" cục bộ và tiến gần hơn về vùng không gian chứa điểm tối ưu toàn cục tại $(0, 0)$, đạt giá trị hàm mục tiêu tốt hơn đáng kể ($f(x) \approx 9.23$).

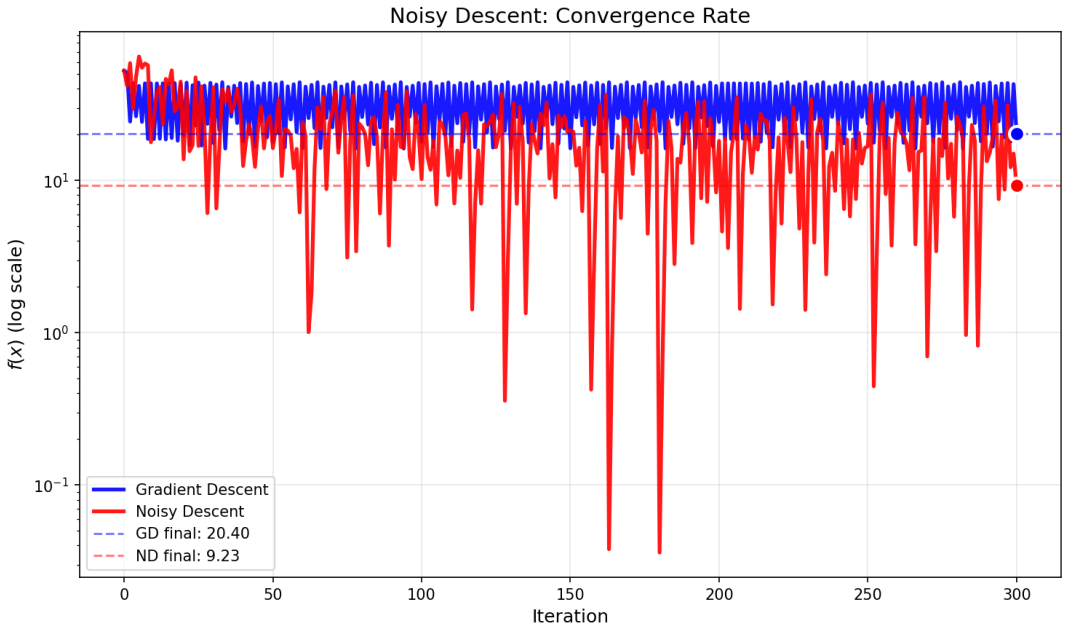
Tốc độ hội tụ: Biểu đồ log-scale cho thấy giá trị hàm mục tiêu của *Gradient Descent* giảm rất nhanh nhưng sớm đi ngang do bị kẹt. Trong khi đó, *Noisy Descent* có sự dao động mạnh trong những vòng lặp đầu tiên do biên độ nhiễu lớn, tạo điều kiện cho việc thăm dò. Khi số vòng lặp tăng lên, nhiễu giảm dần giúp thuật toán ổn định và hội tụ sâu hơn vào vùng cực tiểu tốt hơn.

3.1.3 Kết luận

Noisy Descent giúp cân bằng giữa khả năng thăm dò không gian tìm kiếm và khả năng khai thác điểm tối ưu, điều mà *Gradient Descent* thuần túy thường thất bại trong các bài toán không lồi.



Hình 1: Quỹ đạo tối ưu hóa của Gradient Descent và Noisy Descent trên bề mặt hàm Rastrigin.



Hình 2: Phân tích tốc độ hội tụ của thuật toán Noisy Descent

3.2 Mesh Adaptive Direct Search

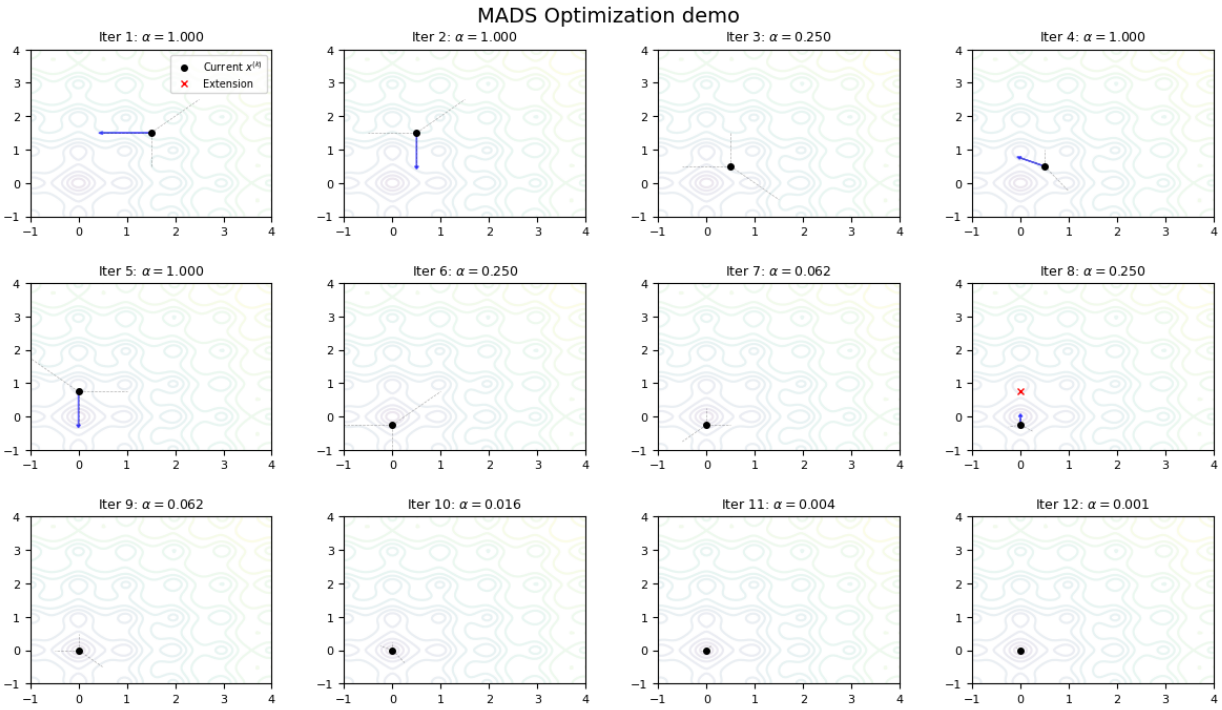
Thực nghiệm được tiến hành trên hàm Ackley là một benchmark dùng để đánh giá các thuật toán tối ưu toàn cục do đặc tính có nhiều cực trị địa phương:

- **Hàm mục tiêu:** $f(x) = -20\exp\left(-0.2\sqrt{0.5\sum x_i^2}\right) - \exp\left(0.5\sum\cos(2\pi x_i)\right) + 20 + e.$
- **Không gian tìm kiếm:** $x \in [-1, 4]^2$
- **Điểm khởi tạo:** $x^{(0)} = [1.5, 1.5]$.
- **Cực tiểu toàn cục lý thuyết:** $x^* = [0, 0]$ với $f(x^*) = 0$.

Thuật toán Mesh Adaptive Direct Search (MADS) được cài đặt giống với lý thuyết đã trình bày. Kết quả từ hình 3 mô phỏng quá trình tối ưu qua 12 vòng lặp cho thấy:

- Giai đoạn thăm dò (iter 1 - iter 4): Thuật toán di chuyển nhanh từ điểm khởi tạo $(1.5, 1.5)$ về phía vùng trũng trung tâm. Tham số lưới duy trì ở mức lớn ($\alpha = 1.0$) cho phép bước nhảy dài, tránh bị bẫy tại các gợn sóng của hàm Ackley.
- Giai đoạn tinh chỉnh và hội tụ (iter 5 - iter 12): Khi tiếp cận gần cực tiểu toàn cục, không gian tìm kiếm bị thu hẹp. Thuật toán tự động giảm kích thước lưới để tìm kiếm chính xác hơn.

Thực nghiệm cho thấy thuật toán MADS hoạt động ổn định trên hàm đa cực trị Ackley. Cơ chế điều chỉnh kích thước lưới linh hoạt kết hợp với hướng tìm kiếm ngẫu nhiên giúp thuật toán cân bằng tốt giữa khả năng tìm kiếm toàn cục (tránh cực trị địa phương ban đầu) và hội tụ cục bộ chính xác tại điểm tối ưu $(0, 0)$.



Hình 3: Quá trình tối ưu hóa MADS trên hàm Ackley qua 12 vòng lặp.

3.3 MeZO

3.3.1 Bài toán Rosenbrock

Để đánh giá hiệu quả của thuật toán tối ưu hóa bậc không tiết kiệm bộ nhớ (*MeZO*), ta thực hiện so sánh với thuật toán *Gradient Descent* truyền thống trên *hàm Rosenbrock*. Đây là một hàm thử nghiệm phi lồi nổi tiếng với hình dạng "thung lũng hẹp", cực tiểu toàn cục nằm tại $(1, 1)$. Bài toán này đòi hỏi bộ tối ưu hóa phải có khả năng điều hướng tốt trong các không gian có độ dốc thay đổi mạnh.

Các tham số thực nghiệm được thiết lập như sau:

- Điểm khởi đầu: $\mathbf{x}^{(0)} = (-1.5, 2.0)$.
- Tốc độ học: $\alpha = 0.001$ cho cả hai thuật toán.
- Hệ số sai phân (với MeZO): $\delta = 0.01$.
- Số vòng lặp: 1200.

3.3.2 Phân tích kết quả thực nghiệm

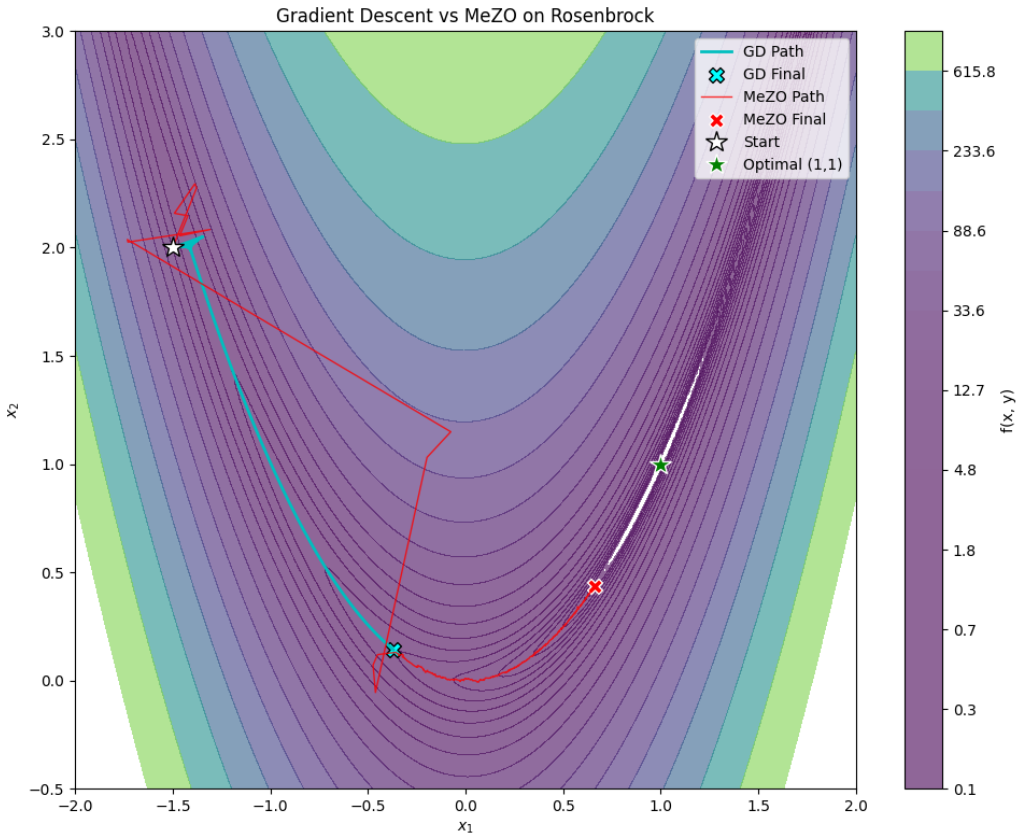
Quỹ đạo di chuyển: Quan sát đồ thị quỹ đạo, phương pháp *Gradient Descent* (đường màu xanh cyan) di chuyển theo một đường cong mượt mà dựa trên thông tin gradient chính xác. Ngược lại, *MeZO* (đường màu đỏ) di chuyển theo hình "răng cưa" hoặc "zigzag". Điều này phản ánh đặc tính của ước lượng bậc không: *MeZO* không biết hướng dốc chính xác mà phải thăm dò thông qua

các hướng nhiễu ngẫu nhiên **z**. Tuy nhiên, *MeZO* vẫn thành công trong việc điều hướng dọc theo lòng thung lũng *Rosenbrock* và tiến gần đến điểm tối ưu toàn cục.

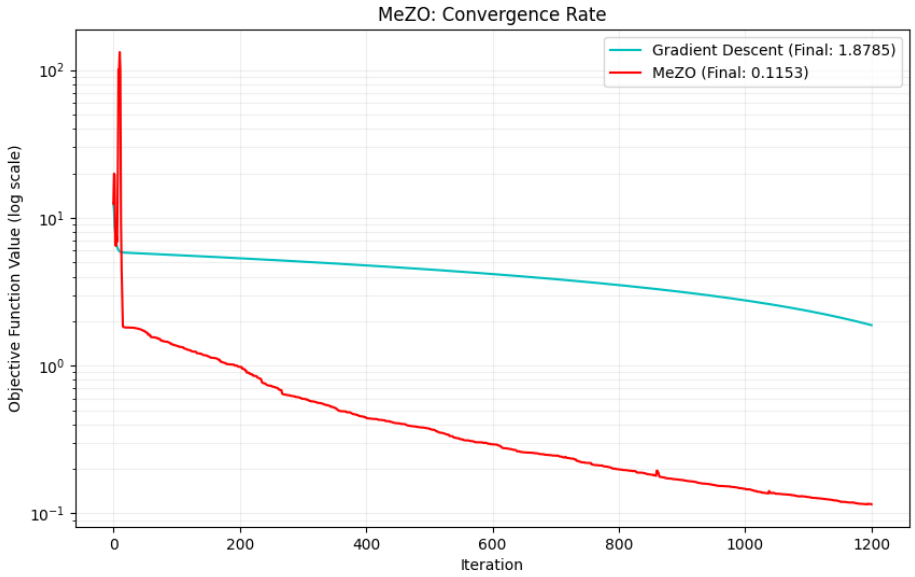
Tốc độ hội tụ: Trong khoảng 50 vòng lặp đầu tiên, giá trị hàm mục tiêu của *MeZO* dao động cực mạnh (biên độ lên tới 10^2) do tính ngẫu nhiên của việc chọn hướng **z**. Tuy nhiên, sau giai đoạn thăm dò này, *MeZO* thiết lập được một lộ trình hội tụ ổn định và vượt xa *Gradient Descent*. Kết thúc 1200 vòng lặp, *Gradient Descent* dừng lại ở mức $f(x) \approx 1.8785$, trong khi *MeZO* đạt được giá trị ấn tượng $f(x) \approx 0.1153$. Điều này chứng minh rằng trong các địa hình "phẳng" như đáy thung lũng Rosenbrock, nơi gradient thực tế rất nhỏ khiến GD hội tụ cực chậm, thì cơ chế nhiễu ngẫu nhiên của *MeZO* lại đóng vai trò như một bộ tạo động lực giúp tìm ra lối thoát tốt hơn.

3.3.3 Kết luận

Thực nghiệm khẳng định *MeZO* không chỉ tiết kiệm bộ nhớ bằng cách loại bỏ việc lưu trữ vectơ gradient mà còn có khả năng hội tụ tốt trên các bề mặt hàm phức tạp. Đây là giải pháp tối ưu cho các hệ thống có tài nguyên tính toán hạn chế hoặc các mô hình quy mô cực lớn mà việc tính toán gradient chính xác là không khả thi.



Hình 4: Quỹ đạo tối ưu hóa của Gradient Descent và MeZO trên bề mặt hàm Rosenbrock



Hình 5: Phân tích tốc độ hội tụ của thuật toán MeZO

3.4 Simulated Annealing

3.4.1 Bài toán Rastrigin

Để minh họa khả năng vượt qua các cực tiểu cục bộ của *Simulated Annealing*, ta thực hiện thử nghiệm so sánh với *Gradient Descent* trên hàm *Rastrigin*. Các tham số thực nghiệm được thiết lập như sau:

- Điểm khởi đầu: $\mathbf{x}^{(0)} = (-2.5, -2.5)$.
- Tham số SA: Nhiệt độ ban đầu $t_0 = 15.0$, độ lệch chuẩn bước nhảy $\sigma = 0.5$, hệ số hạ nhiệt $\gamma = 0.99$.
- Tham số GD: Tốc độ học $\alpha = 0.01$.
- Số vòng lặp: 400.

3.4.2 Phân tích kết quả thực nghiệm

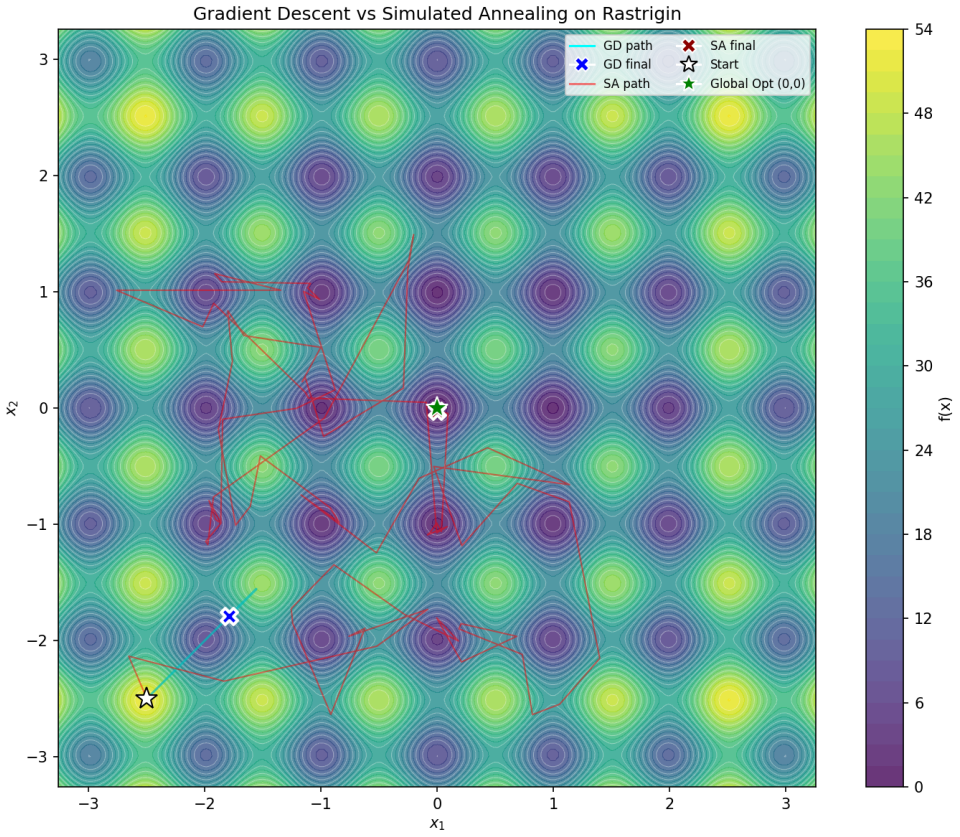
Quỹ đạo di chuyển: Thuật toán *Gradient Descent* (đường màu xanh) nhanh chóng rơi vào cực tiểu cục bộ gần nhất tại vị trí khoảng $(-1.8, -1.8)$ và bị kẹt hoàn toàn tại đó. Ngược lại, quỹ đạo của *Simulated Annealing* (đường màu đỏ) thể hiện một quá trình "thăm dò" rộng khắp. Nhờ cơ chế chấp nhận các bước đi làm tăng giá trị hàm mục tiêu khi nhiệt độ còn cao, SA đã vượt qua hàng loạt các "rào cản" địa hình để tiến về vùng trung tâm và tiếp cận điểm tối ưu toàn cục $(0, 0)$.

Tốc độ hội tụ: Biểu đồ hội tụ cho thấy sự khác biệt rõ rệt về chiến lược tìm kiếm. Đường biểu diễn của *Gradient Descent* nhanh chóng đi ngang và dao động liên tục ở mức cao ($f(x) \approx 21.3079$). Trong khi đó, đường biểu diễn của *Simulated Annealing* có dạng hình bậc thang đi xuống: thuật

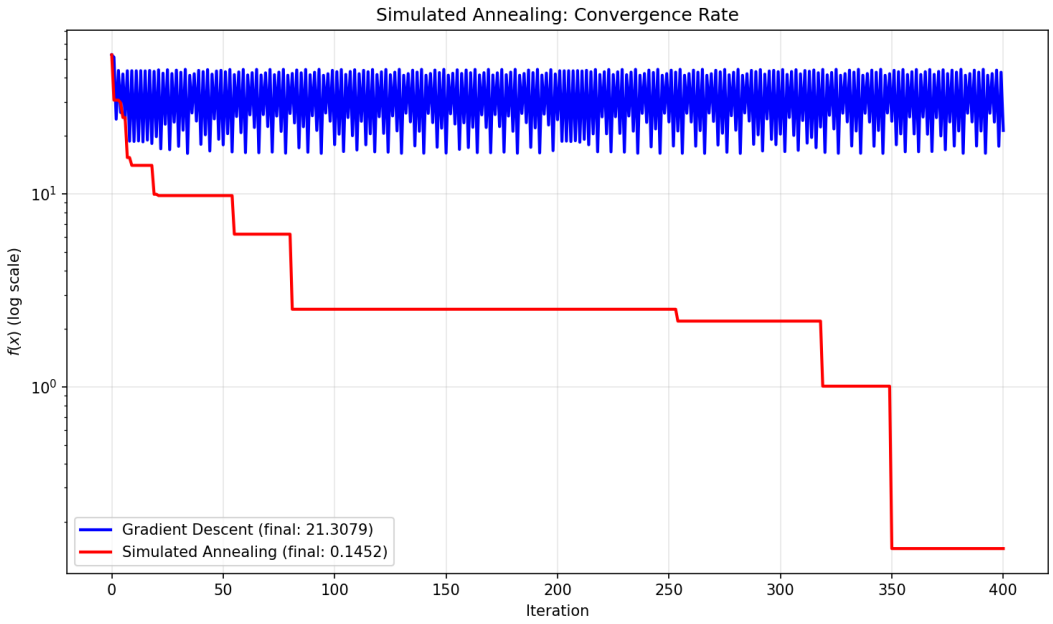
toán có những giai đoạn duy trì giá trị cũ để thăm dò vùng lân cận, sau đó đột ngột nhảy sang một "hố" thấp hơn. Kết thúc 400 vòng lặp, SA đạt được giá trị hàm mục tiêu ấn tượng $f(x) \approx 0.1452$, tiệm cận rất gần điểm tối ưu toàn cục.

3.4.3 Kết luận

Thực nghiệm cho thấy *Simulated Annealing* là một công cụ cực kỳ mạnh mẽ cho các bài toán tối ưu hóa không lồi (non-convex) có bề mặt năng lượng gồ ghề. Mặc dù tốc độ hội tụ có thể chậm hơn các phương pháp dựa trên gradient trong các hàm đơn giản, nhưng tính ngẫu nhiên được kiểm soát bởi nhiệt độ giúp SA trở thành giải pháp cứu cánh để thoát khỏi các cực tiểu cục bộ và tìm đến điểm tối ưu toàn cục trong không gian tìm kiếm phức tạp.



Hình 6: Quỹ đạo tối ưu hóa của Gradient Descent và Simulated Annealing trên bề mặt hàm Rastrigin



Hình 7: Phân tích tốc độ hội tụ của thuật toán Simulated Annealing

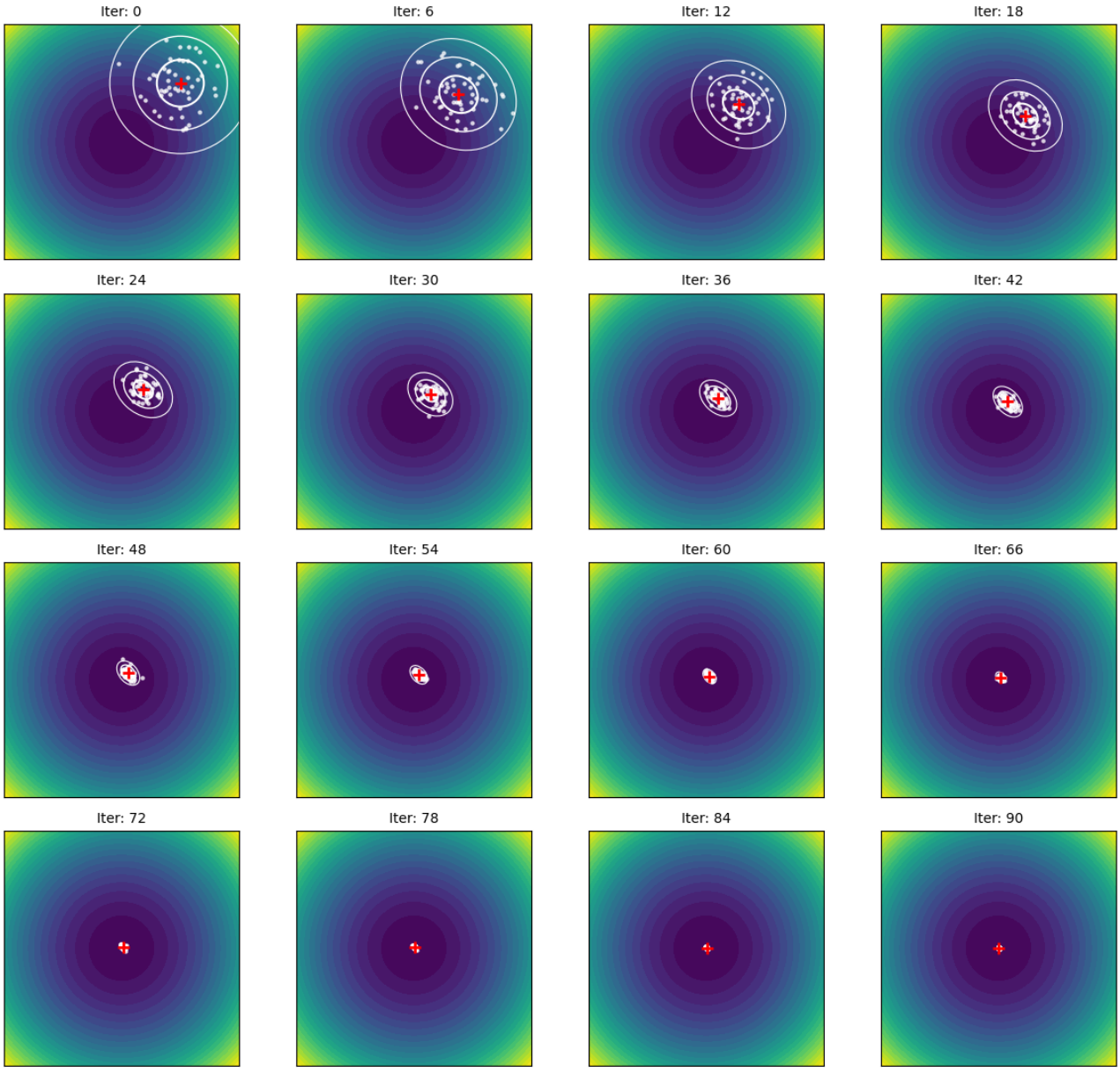
3.5 Cross-Entropy Method

Thực nghiệm được tiến hành trên hàm Sphere. Đây là một hàm kiểm thử lồi (convex) điển hình, bề mặt trơn và đơn điệu. Bài toán này được sử dụng làm cơ sở để đánh giá tính đúng đắn của thuật toán và tốc độ hội tụ về điểm tối ưu.

- **Hàm mục tiêu:** $f(\mathbf{x}) = \sum_{i=1}^{d} x_i^2$
- **Không gian tìm kiếm:** $x \in [-10.00, 10.00]$
- **Điểm khởi tạo:** $\mu^{(0)} = [5.0, 5.0], \Sigma^{(0)} = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$
- **Cực tiểu toàn cục lý thuyết:** $x^* = [0, 0]$ với $f(x^*) = 0$.

Thuật toán CEM được cài đặt với thông số như sau:

- **Kích thước quần thể:** $m = 50$
- **Số bước lặp tối đa:** $k_{max} = 100$
- **Số lượng mẫu ưu tú:** $m_{elite} = 5$.
- **Hệ số làm mịn:** $\alpha = 0.9$.



Hình 8: Quá trình tối ưu hóa CEM trên hàm Sphere.

Nhận xét kết quả từ hình 8:

- Tại bước lặp đầu tiên, các mẫu được phân bố rộng khắp xung quanh điểm khởi tạo $(5.0, 5.0)$. Các đường elip có bán kính lớn, cho thấy phương sai ban đầu cao, đảm bảo thuật toán bao phủ một vùng không gian đủ rộng để tìm kiếm các mẫu tốt.
- Dưới tác động của phương pháp chọn m_{elite} mẫu tốt nhất và cập nhật MLE, vector trung bình μ bắt đầu dịch chuyển dần về phía gốc tọa độ $(0, 0)$. Do sử dụng hệ số làm mịn cao $(\alpha = 0.9)$, sự thay đổi của các đường elip diễn ra rất mượt mà và ổn định qua từng vòng lặp.

Thuật toán không bị hiện tượng co cụm đột ngột hay mất tính đa dạng quá nhanh, giúp quỹ đạo hội tụ tránh được hiện tượng hội tụ sớm.

- Trong giai đoạn cuối, các mẫu tập trung mật độ cao ngay tại điểm cực trị $(0, 0)$. Các đường elip co lại thành các điểm rất nhỏ, cho thấy ma trận hiệp phương sai Σ đã tiệm cận ma trận không. Tại Iter 90, thuật toán đạt được độ chính xác rất cao, chứng minh rằng việc kết hợp giữa MLE và kỹ thuật làm mịn tham số giúp CEM vừa đảm bảo tốc độ hội tụ, vừa duy trì được sự ổn định trong các bài toán tối ưu liên tục.

3.6 Natural Evolution Strategies

Thực nghiệm được tiến hành trên hàm Wheeler's Ridge. Đây là một hàm có địa hình dạng thung lũng cong và sâu, rất khó đối với các thuật toán tối ưu hóa thông thường do sự tương quan mạnh giữa các biến, các bước nhảy cần phải đi dọc theo đáy thung lũng thay vì trượt xuống sườn dốc:

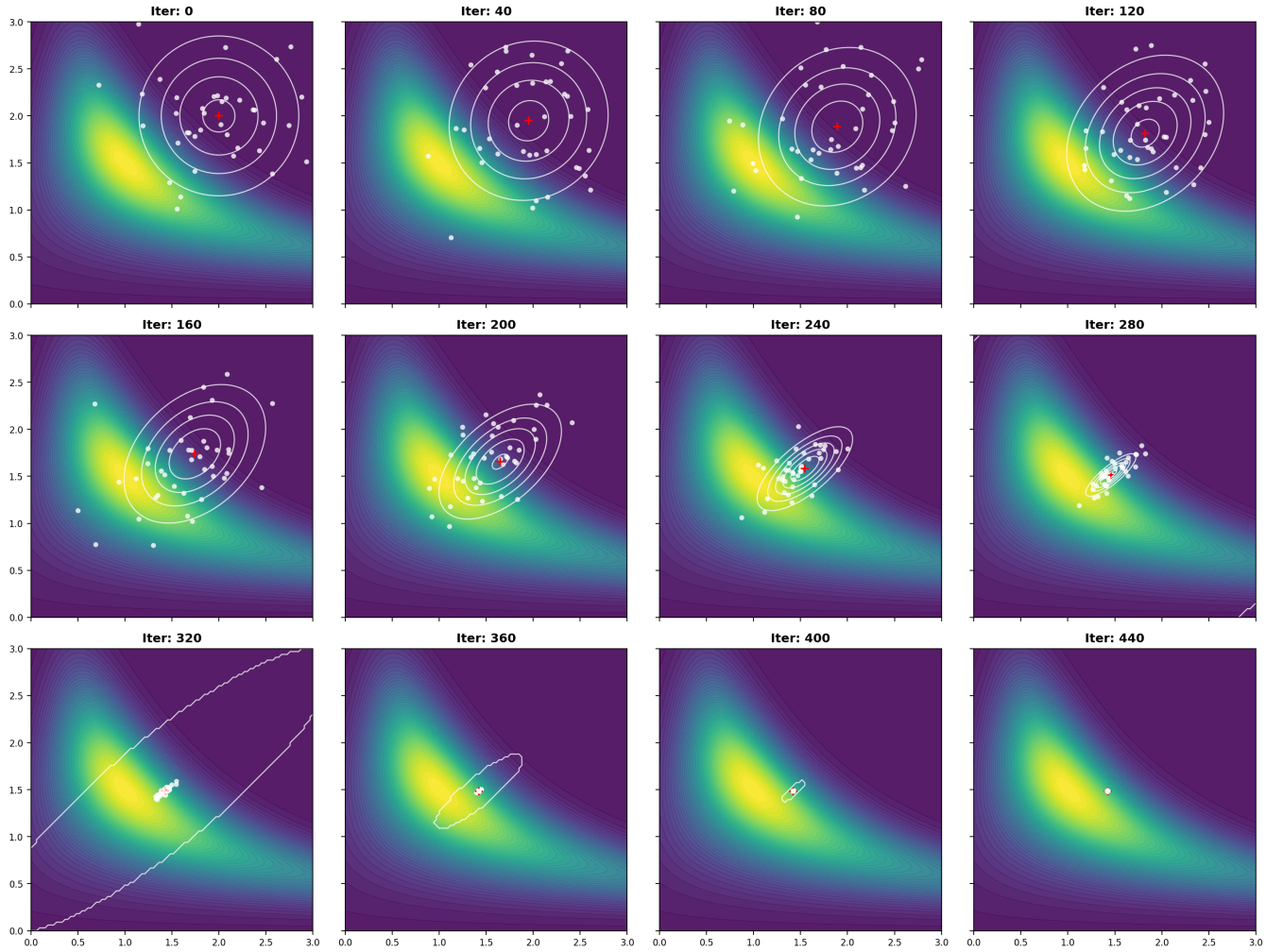
- **Hàm mục tiêu:** $f(\mathbf{x}) = -\exp\left(-(x_1x_2 - a)^2 - (x_2 - a)^2\right)$, với tham số $a = 1.5$
- **Không gian tìm kiếm:** $x \in [0, 3]^2$.
- **Khởi tạo:** $\boldsymbol{\mu}^{(\mathbf{0})} = [2.0, 2.0], \mathbf{A} = 0.5\mathbf{I} \Rightarrow \Sigma^{(k)} = 0.25\mathbf{I}$.
- **Cực tiểu toàn cục lý thuyết:** $x^* = [1, 1.5]$ với $f(x^*) = -1$.

Thuật toán NES được cài đặt với thông số như sau:

- **Kích thước quần thể:** $m = 40$
- **Số bước lặp tối đa:** $k_{max} = 450$
- **Tốc độ học:** $\alpha = 0.01$.

Nhận xét kết quả từ hình 9:

- **Giai đoạn thăm dò (Iter 0 - 120):** Vectơ trung bình bắt đầu di chuyển từ vị trí khởi tạo $[2.0, 2.0]$ về phía dốc của "sống núi" (ridge).
- **Giai đoạn tinh chỉnh (Iter 160 - 280):** Ma trận hiệp phương sai thay đổi hình dạng rõ rệt. Các đường elip kéo dài để khớp với độ cong của thung lũng, giúp các mẫu tập trung dọc theo vùng có giá trị hàm mục tiêu tốt hơn.
- **Giai đoạn hội tụ (Iter 320 - 440):** Sau khi tìm được điểm cực trị, ma trận hiệp phương sai giảm phương sai, co nhỏ lại làm cho các mẫu tập trung dày đặc quanh điểm cực trị $[1.0, 1.5]$. Tại iter 440, phân phối gần như đạt trạng thái hội tụ với độ chính xác cao.



Hình 9: Quá trình tối ưu hóa NES trên hàm Wheeler's Ridge.

Quan sát thấy tại bước lặp 320, ma trận hiệp phương sai có sự giãn nở đột ngột mặc dù mật độ của các mẫu tập trung quanh điểm cực trị. Đây là hiện tượng bùng nổ ma trận hiệp phương sai, làm mất ổn định của phân phối tìm kiếm. Nguyên nhân do tại đỉnh hàm Wheeler:

- Giá trị hàm mục tiêu $f(x)$ tiệm cận -1, do đó bề mặt hàm trở nên gần như phẳng, ta có thể coi $f(x^{(i)}) \approx -1$.
- Vì các mẫu có mật độ tập trung rất sát giá trị $\boldsymbol{\mu}$, nên hiệu số $(x^{(i)} - \boldsymbol{\mu})$ tiến dần về 0.
- Gradient của log-likelihood theo ma trận hiệp phương sai:

$$\nabla_{\boldsymbol{\Sigma}} \log p \approx \frac{1}{2} \left(\boldsymbol{\Sigma}^{-1} \underbrace{(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^{\top}}_{\approx \mathbf{0}} \boldsymbol{\Sigma}^{-1} - \boldsymbol{\Sigma}^{-1} \right) \approx -\frac{1}{2} \boldsymbol{\Sigma}^{-1}$$

- Ước lượng Gradient tổng thể:

$$\nabla_{\boldsymbol{\Sigma}} J \approx \frac{1}{m}\sum_{i=1}^{m} \underbrace{f(\mathbf{x}^{(i)})}_{\approx -1} \cdot \underbrace{\nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x}^{(i)})}_{\approx -\frac{1}{2}\boldsymbol{\Sigma}^{-1}} \approx \frac{1}{m}\sum_{i=1}^{m}(-1)\left(-\frac{1}{2}\boldsymbol{\Sigma}^{-1}\right) = \frac{1}{2}\boldsymbol{\Sigma}^{-1}$$

Như vậy gradient của ma trận hiệp phương sai bị chi phối bởi thành phần nghịch đảo Σ^{-1}. Việc cập nhật với tốc độ học $\boldsymbol{\alpha}$ cố định tại vùng này gây ra một bước nhảy quá đà (overshoot) về kích thước phân phối. Tuy nhiên, vì vector trung bình $\boldsymbol{\mu}$ đang nằm gần cực trị nên các mẫu được tạo ở bước sau sẽ kéo ma trận hiệp phương sai co lại rất nhanh về điểm cực trị (Iter 360).

3.7 Covariance Matrix Adaptation

Thực nghiệm được tiến hành trên hàm Flower mô phỏng địa hình đa cực trị với các thung lũng hẹp hướng tâm, nhằm đánh giá khả năng thích nghi của ma trận hiệp phương sai:

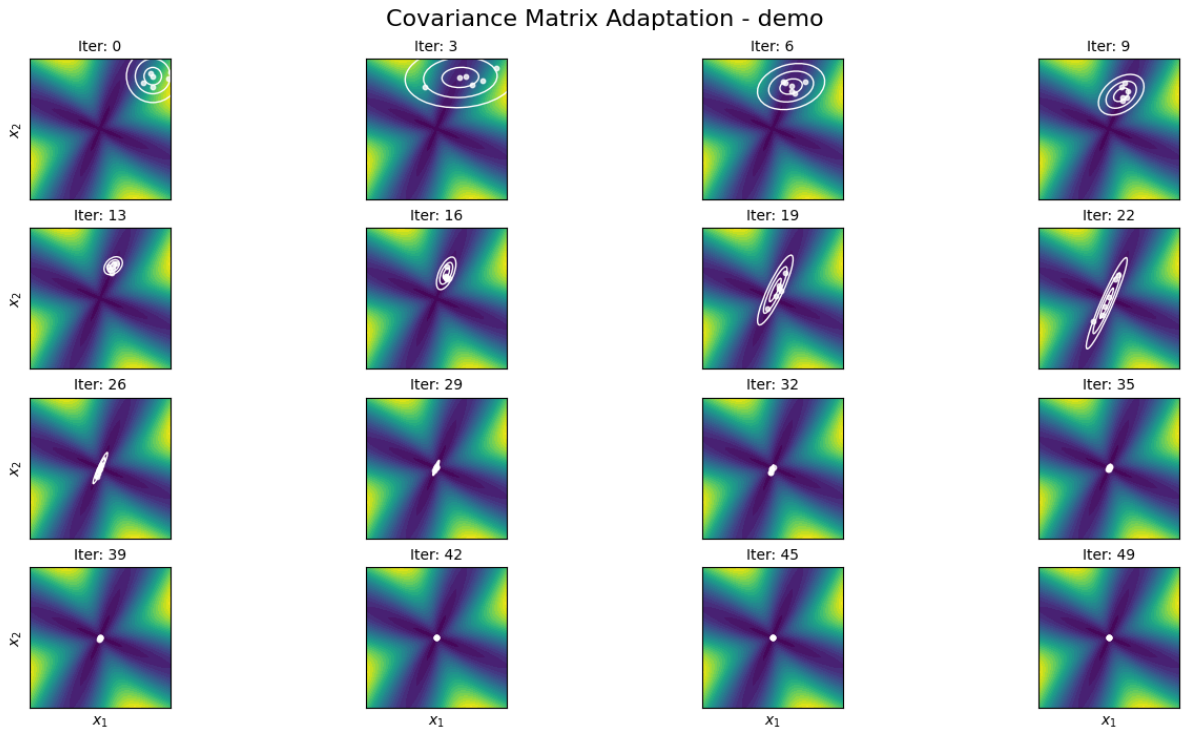
- **Hàm mục tiêu:** $f(x) = r + 0.8r\sin(4\theta)$ với $r = ||x||$ và $\theta = \arctan(x_2/x_1)$.
- **Không gian tìm kiếm:** $x \in [-4, 4]^2$.
- **Điểm khởi tạo:** $x^{(0)} = [3.0, 3.0]$ với độ lệch chuẩn bước nhảy ban đầu $\sigma = 0.5$.
- **Cực tiểu toàn cục lý thuyết:** $x^* = [0, 0]$ với $f(x^*) = 0$.

Thuật toán CMA-ES được cài đặt với các thông số sau:

- **Kích thước quần thể:** Được tính toán tự động dựa trên số chiều ($n = 2$) theo công thức $m = 4 + \lfloor 3\ln(n) \rfloor$.
- **Độ lệch chuẩn bước nhảy ban đầu (σ):** 0.5.
- **Số bước lặp tối đa:** 50.
- **Hằng số:** Sử dụng các hằng số được khuyến nghị trong tài liệu.

Nhận xét kết quả từ hình 10:

- **Giai đoạn thăm dò (iter 0 - 9):** Tại iter 0, quần thể phân bố đẳng hướng quanh điểm khởi tạo $(3.0, 3.0)$. Đến iter 9, thuật toán bắt đầu nhận diện độ dốc, dịch chuyển dần về phía thung lũng của hàm số.
- **Giai đoạn thích nghi ma trận hiệp phương sai (iter 13 - 22):** Các đường elip bao quanh quần thể bị kéo dãn đáng kể và xoay trục dọc theo hướng thung lũng hẹp. Điều này chứng tỏ ma trận Σ đã học được cấu trúc của hàm mục tiêu để tăng bước nhảy theo hướng có triển vọng.
- **Giai đoạn tinh chỉnh và hội tụ (iter 26 - 49):** Từ iter 26, khi tiếp cận gần vùng cực tiểu toàn cục, kích thước bước nhảy σ giảm dần. Tại iter 49, quần thể co cụm chặt chẽ tại toạ độ $(0, 0)$, đạt trạng thái hội tụ với độ chính xác cao.



Hình 10: Quá trình tối ưu hóa CMA-ES trên hàm Flower.

4 Các nghiên cứu hiện đại liên quan đến phương pháp tối ưu hóa ngẫu nhiên

Sự phát triển của tối ưu hóa ngẫu nhiên không chỉ dừng lại ở các bài toán lồi đơn giản mà đã mở rộng thành hai nhánh chủ yếu: (1) các phương pháp tiếp cận dựa trên *trí tuệ bầy đàn* để giải quyết các không gian tìm kiếm phức tạp, không có đạo hàm; và (2) các thuật toán tối ưu hóa dựa trên *gradient thế hệ mới* nhằm vượt qua giới hạn về tài nguyên phần cứng trong huấn luyện mô hình lớn.

4.1 Particle Swarm Optimization

Particle Swarm Optimization [3] (Tối ưu hóa bầy hạt) là một thuật toán metaheuristic được giới thiệu bởi Kennedy và Eberhart vào năm 1995, lấy cảm hứng từ hành vi xã hội của đàn chim hoặc đàn cá.

- **Cơ chế hoạt động:** PSO duy trì một quần thể các thực thể gọi là "hạt" (particles). Mỗi hạt đại diện cho một lời giải tiềm năng trong không gian tìm kiếm. Các hạt di chuyển dựa trên hai yếu tố chính: vị trí tốt nhất mà chính nó từng đạt được (p_{best}) và vị trí tốt nhất mà toàn bộ bầy đàn từng đạt được (g_{best}).

- **Ưu điểm:** Đây là phương pháp tối ưu hóa không cần đạo hàm, giúp nó cực kỳ hiệu quả đối với các hàm mục tiêu không lồi, đứt đoạn hoặc có bề mặt phức tạp nơi mà gradient không khả dụng.

- **Nghiên cứu hiện đại:** Hiện nay, PSO vẫn được nghiên cứu mạnh mẽ thông qua các biến thể lai (Hybrid PSO) kết hợp với các thuật toán học máy để tối ưu hóa siêu tham số hoặc giải quyết các bài toán tối ưu hóa đa mục tiêu trong kỹ thuật điện và logistics.

4.2 GaLore: Gradient Low-Rank Projection

GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection [4] là một nghiên cứu đột phá được công bố vào năm 2024 bởi Zhao và các cộng sự nhằm giải quyết rào cản về tài nguyên phần cứng khi huấn luyện các mô hình ngôn ngữ lớn.

- **Bối cảnh:** Thông thường, các bộ tối ưu hóa như Adam yêu cầu bộ nhớ gấp 2-3 lần kích thước tham số mô hình để lưu trữ các trạng thái. Điều này khiến việc huấn luyện mô hình 7B tham số trên GPU tiêu dùng là không thể.
- **Cơ chế:** Thay vì nén trọng số như các phương pháp LoRA, GaLore tập trung vào việc nén *gradient*. Thuật toán sử dụng phép chiếu thứ bậc thấp (Low-rank projection) để chuyển gradient sang một không gian có chiều dữ liệu nhỏ hơn. Sau khi cập nhật, nó lại được chiếu ngược lại không gian ban đầu.
- **Đóng góp:** GaLore cho phép giảm bộ nhớ optimizer lên tới 65.5% mà không làm giảm độ chính xác của mô hình. Đây là cột mốc quan trọng giúp dân chủ hóa việc huấn luyện AI, cho phép huấn luyện các mô hình lớn trên các phần cứng phổ thông như NVIDIA RTX 4090.

4.3 SOAP: Improving and Stabilizing Shampoo using Adam

SOAP [5] là một nghiên cứu đột phá về tối ưu hóa ngẫu nhiên bậc cao, được đề xuất bởi nhóm nghiên cứu tại Đại học Harvard và Viện Kempner vào cuối năm 2024. Thuật toán này đại diện cho thế hệ các bộ tối ưu hóa tiền điều kiện nhằm giải quyết bài toán hội tụ trên mô hình ngôn ngữ lớn.

- **Cơ chế hoạt động:** SOAP kết hợp sức mạnh của Shampoo (một phương pháp tiền điều kiện bậc cao) và AdamW. SOAP thực hiện thuật toán Adam bên trong không gian riêng (eigenbasis) của ma trận tiền điều kiện Shampoo. Thay vì cập nhật gradient trong không gian tọa độ gốc, SOAP xoay gradient sang không gian eigenbasis, thực hiện cập nhật thích nghi, rồi xoay ngược trở lại.
- **Tính chất ngẫu nhiên:** Tương tự như SGD hay Adam, SOAP hoạt động dựa trên việc lấy mẫu mini-batch ngẫu nhiên (B_t) từ tập dữ liệu. Tuy nhiên, nó vượt trội hơn các phương pháp bậc một nhờ khả năng ước lượng độ cong của bề mặt hàm mất mát thông qua các ma trận tiền điều kiện L và R được cập nhật liên tục theo thời gian.
- **Hiệu suất vượt trội:** Nghiên cứu thực nghiệm cho thấy SOAP giúp giảm số lượng bước lặp hơn 40% và rút ngắn thời gian thực tế hơn 35% so với AdamW trong quá trình pre-training cho LLMs. Ngoài ra, SOAP giải quyết được vấn đề mất ổn định của Shampoo khi tần suất cập nhật tiền điều kiện thấp, giúp quá trình tối ưu hóa diễn ra mượt mà hơn.

5 Kết luận, đánh giá

5.1 Kết luận

Thông qua quá trình nghiên cứu và thực hiện đồ án Stochastic Methods, nhóm đã đạt được những kết quả quan trọng sau:

- Nắm được bản chất, cách cài đặt các thuật toán tối ưu hóa ngẫu nhiên.
- Trình bày một cách có hệ thống, chi tiết và minh họa về các thuật toán.

5.2 Đánh giá

Đối với đồ án này, nhóm tự đánh giá **hoàn thành tốt và đầy đủ** các yêu cầu đã được đặt ra:

- Trình bày đầy đủ và chi tiết các thuật toán liên quan đến phương pháp tối ưu hóa ngẫu nhiên.
- Cài đặt code và minh họa các thuật toán.
- Mở rộng, tìm hiểu các nghiên cứu hiện đại liên quan đến phương pháp tối ưu hóa ngẫu nhiên.

Tài liệu

[1] Anh Khoa, Hoang Trung, and Duc Tai. github/anhKhoa585/AI-Fundamentals-Proj3. https://github.com/anhKhoa585/AI-Fundamentals-Proj3, 2025.

[2] Mykel J Kochenderfer and Tim A Wheeler. *Algorithms for optimization*. Mit Press, 2019.

[3] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, volume 4, pages 1942–1948. ieee, 1995.

[4] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. Galore: Memory-efficient llm training by gradient low-rank projection, 2024.

[5] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. Soap: Improving and stabilizing shampoo using adam, 2025.